Name : Nivasini V.M

Regno : 192511356

Course : CSA 0613 DESIGN AND ANALYSIS OF ALGORITHM.

88.1

CO2 - AT-1

3. BINARY SEARCH - LAST ELEMENT ANALYSIS

AIM

To find the element 30 in the sorted dataset [2, 6, 10, 14, 18, 22, 26, 30] using the binary search algorithm and analyze its performance.

PROBLEM UNDERSTANDING

The given dataset is

[2, 6, 10, 14, 18, 22, 26, 30]

Target element = 30

Since the data is already sorted in ascending order, Binary search is the most suitable searching technique. Instead of checking every element one by one, it repeatedly divides the search space into two halves until the target is found.

METHOD SELECTION

Algorithm used : Binary Search.

Reason for selection.

* The dataset is sorted
* Binary search reduces the search space by half after every comparison.
* It is much faster than linear search for large datasets.

ALGORITHM

1. Initialize
   - low = 0
   - high = n-1
2. Calculate the middle index :
   - mid = (low + high) // 2
3. Compare the middle element with the target
4. If the target is equal to the middle element, return the index.
5. If the target is greater, search the right half.
6. If the target is smaller, search the left half.
7. If the element is found or low > high repeat it.

STEP BY STEP SOLUTION

Dataset

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | |
|---|---|---|---|---|---|---|---|---|---|
| Value | 2 | 6 | 10 | 14 | 18 | 22 | 26 | 30 | |

Target = 30

Iteration 1

- Low = 0
- High = 7
- Mid = (0+7)//2 = 3.
- Element = 14

Comparison

- 30 > 14

Search the right half.

New values:

Low = 4

High = 7

Iteration 2

- Low = 4
- High = 7
- Mid = (4+7)//2 = 5
- Element = 22.

Comparison

- 30 > 22.

Search the right half

New Values.

- Low = 6
- High = 7

Iteration 3

- Low = 6
- High = 7
- Mid = (6+7)//2 = 6
- Element = 26

Comparison

- 30 > 26

Search the right half

NEW Values

Low = 7

High = 7.

Iteration 4

Low = 7

High = 7

Mid = (7+7)//2 = 7

Element = 30

Comparison

30 = 30

Element found

CALCULATIONS.

| Comparison | Middle Element | Result. |
|---|---|---|
| 1 | 14 | Target is greater |
| 2 | 22 | Target is greater |
| 3 | 26 | Target is greater |
| 4 | 30 | Element found. |

Total comparison = 4

Index of 30 = 7

PYTHON CODE

```
arr = [2, 6, 10, 14, 18, 22, 26, 30]

x = 30

l, h = 0, len(arr) - 1

while l <= h:

    m = (l + h) // 2

    if arr[m] == x:

        print("Element found at index", m)

        break

    elif arr[m] < x:

        l = m + 1

    else:

        h = m - 1
```

OUTPUT

Element found at index 7.

TIME COMPLEXITY

Best case : O(1) (target found at first middle element)

Average case : O(log n)

Worst case : O(log n)

SPACE COMPLEXITY

O(1) (Iterative implementation)

RESULT

The target element 30 was successfully found at index 7 after 4 comparisons using binary search

INTERPRETATION OF RESULTS.

Although the target is located at last position in array, binary search does not examine every element. Instead of, it repeatedly halves the search space, requiring only 4 comparison to locate the element in array of 8 elements. Compared to linear search, which could require upto 8 comparison binary search provides a significantly faster.